

UAV Swarm

Mukund Thakur

Roll Number: 231010230

Department of Electronics and
Communication Engineering

International Institute of Information
Technology, Naya Raipur

Sahil Kaushik

Roll Number: 231010240

Department of Electronics and
Communication Engineering

International Institute of Information
Technology, Naya Raipur

Satyendra Kumar Tandan

Roll Number: 231010242

Department of Electronics and
Communication Engineering

International Institute of Information
Technology, Naya Raipur

Abstract—This research paper presents the design and implementation of a cooperative multi-drone system capable of executing autonomous formation control and task-specific behaviors, such as square formation flying and bomb-dropping. Using the CoppeliaSim simulation environment, we developed and tested a system where multiple quadrotor drones synchronize movements and maintain geometric configurations while adapting to controller input and dynamic changes in the environment. The simulation validates the efficacy of centralized and distributed control strategies for precise coordination of UAVs (Unmanned Aerial Vehicles) in real-time. Our work emphasizes the integration of control algorithms with real-world constraints such as inter-drone communication, delay handling, and precise actuator control. The system demonstrates the feasibility of scalable drone swarming operations for surveillance, disaster response, and defense applications.

Index Terms—drone formation, quadrotor, swarm control, autonomous UAVs, bomb dropping, CoppeliaSim, multi-agent systems, simulation, control systems

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs), commonly referred to as drones, have emerged as pivotal tools in a wide array of applications, ranging from surveillance and mapping to environmental monitoring and defense operations. As technological capabilities expand, the concept of autonomous UAV swarms has gained significant attention in both academia and industry. These swarms mimic the collective behavior found in nature—such as flocks of birds or schools of fish—and apply it to coordinated aerial systems.

This project focuses on simulating a swarm of UAVs that operate autonomously in a coordinated square formation while executing dynamic payload operations such as bomb-dropping. The goal is to replicate real-world mission scenarios like precision strikes or coordinated deliveries in a controlled, simulated environment.

The simulation is carried out using CoppeliaSim (formerly V-REP), which provides a high-fidelity environment for real-time physics and scripting. Each UAV is modeled with realistic dynamics and controlled through Python scripts interfaced with the CoppeliaSim Remote API. The central challenge addressed is maintaining a stable formation and synchronized payload deployment, even in the presence of communication delays or potential leader failure.

II. SYSTEM DESIGN AND IMPLEMENTATION

A. Simulation Environment

CoppeliaSim was selected as the simulation platform due to its advanced physics engine, support for distributed control, and seamless integration with external APIs. It allows flexible control through embedded Lua scripts and supports remote programming in Python, making it ideal for multi-agent robotic systems.

Each UAV in the simulation is modeled as a quadrotor with six degrees of freedom (DOF), enabling full control over translation and rotation in 3D space. The simulation environment includes gravity, drag, collision detection, and sensor emulation to approximate real-world behavior.

B. Drone Modeling

The UAVs were designed to encapsulate both physical and computational realism. Each drone features:

- **Propeller and Thrust Modeling:** Simulated brushless motors generate thrust based on torque-speed relationships, with control over individual rotor speeds.
- **PID-Based Flight Stabilization:** Individual PID controllers manage roll, pitch, yaw, and altitude stability, enabling smooth motion and attitude control.
- **Communication Scripts:** Lua scripts on each drone handle event triggers, sensor updates, and inter-agent communication using remote signals.
- **Synchronization Modules:** Python-based external control scripts coordinate formation logic and command propagation between agents.

C. Formation Control Algorithm

A square formation was adopted for its simplicity and scalability. The swarm is led by a designated leader drone whose position serves as a dynamic reference for all follower drones. The relative positions of followers are maintained by calculating offset vectors from the leader's current location.

The position of the i -th follower drone is computed as:

$$P_i = P_L + \Delta x_i \hat{i} + \Delta y_i \hat{j} \quad (1)$$

where P_L is the position vector of the leader, and $\Delta x_i, \Delta y_i$ are the fixed formation offsets along the X and Y axes.

To ensure dynamic responsiveness, a consensus-based control protocol is implemented. Each follower periodically updates its setpoint based on the leader's broadcasted coordinates and applies PID corrections to minimize deviation. The decentralized computation enhances robustness against latency and partial communication failures.

D. Bomb-Dropping Mechanism

The UAVs are equipped with a simulated payload and a sensor-driven actuation mechanism. A downward-facing ray sensor continuously measures the vertical distance to the target. When the UAV reaches a predefined threshold height over the target, a Lua script triggers the release mechanism by actuating a revolute joint.

The simplified control logic is represented as follows:

```
if distance_to_target < threshold:
    sim.setJointTargetPosition(bomb_joint, 1)
```

The payload is attached via a joint that, once triggered, disengages to simulate a drop. This mechanism ensures accurate deployment during flight without requiring the UAV to hover.

III. CONTROL ALGORITHMS

For flight stabilization and trajectory correction, we employed a Proportional-Integral-Derivative (PID) control strategy. The controller calculates the necessary actuation based on positional error as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2)$$

where $e(t)$ denotes the current error in altitude, position, or orientation, and K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively.

Each follower drone determines its target position from the formation algorithm and corrects its trajectory using a PID loop. This feedback loop runs in real time and provides stability under dynamic disturbances, such as sudden velocity changes or wind drag simulated in the environment.

In future expansions, more adaptive controllers like Linear Quadratic Regulators (LQR) or Model Predictive Controllers (MPC) can be integrated to handle more complex swarm behaviors and uncertainty.

IV. TESTING AND RESULTS

A. Test Scenarios

The simulation system was tested under various mission-critical scenarios to evaluate the robustness, accuracy, and responsiveness of the UAV swarm:

- **Coordinated Square Formation Flight:** Drones initiated and maintained a square pattern throughout the flight with dynamic real-time updates.
- **Precision Bomb-Dropping on Static Targets:** Each UAV was tasked with payload release over a known, fixed ground coordinate.

- **Leader Failure Response:** Mid-flight failure of the leader drone was simulated to observe reformation and stability among followers.
- **Communication Delay Tolerance:** Randomized latency (50–300 ms) was introduced to test system adaptability and control loop resiliency.

B. Performance Metrics

The following table summarizes the key performance indicators captured during the test scenarios:

TABLE I
SWARM SYSTEM PERFORMANCE SUMMARY

Metric	Observed Value
Average Formation Deviation	3.7% from intended grid
Drop Accuracy (measured from target center)	92.3% success rate (± 0.5 m)
Controller Response Time	0.12 seconds
Stabilization Error (mean across trials)	0.19 meters
Leader Failure Recovery Time	< 2.1 seconds
Communication Latency Tolerance	Up to 300 ms without major drift
CPU Utilization (4 UAVs)	34.5% on Intel i5 11th Gen

C. Visual Confirmation

Below is a summary of the contributors to the simulation study:

D. Observations

- The UAVs exhibited stable flight characteristics under normal conditions with minimal deviation from designated trajectories.
- Payload drops were precise in over 90
- The leader-follower logic allowed seamless handover when the leader drone was disabled or removed from the swarm.
- Communication delays up to 300 milliseconds were tolerated with marginal drift; however, beyond this threshold, positional errors started accumulating.
- The formation held robustly under moderate wind disturbance simulations (± 2 m/s), although altitude fluctuations slightly increased.

V. CHALLENGES AND FUTURE WORK

A. Challenges Faced

- **Time Synchronization:** CoppeliaSim's simulation ticks needed to be aligned precisely with control commands from Python to avoid command repetition or dropouts.
- **Real-Time Coordination:** Achieving synchronous movement across multiple drones while compensating for variable computation delays was technically challenging.
- **Controller Tuning:** PID gains required individual tuning for roll, pitch, yaw, and altitude for each drone, consuming significant iteration cycles.
- **Sensor Latency Simulation:** Creating realistic sensor delays in simulation to mimic real-world latency without compromising stability.

B. Planned Enhancements

- **Advanced Controllers:** Integrate Linear Quadratic Regulator (LQR) or Model Predictive Control (MPC) for optimal path planning and stability in dynamic environments.
- **Decentralized Architecture:** Move towards a peer-to-peer control scheme where drones make decisions independently with minimal dependency on a central node.
- **Obstacle Avoidance:** Add environment-aware features using simulated LIDAR or vision sensors to facilitate real-time navigation.
- **Scalability Testing:** Extend the simulation to support 10–20 UAVs and assess performance degradation under scale.
- **Hardware Integration:** Port the control algorithms to run on actual embedded systems such as Raspberry Pi or Pixhawk for physical deployment.

VI. CONCLUSION

This project successfully demonstrates the simulation of an autonomous UAV swarm capable of maintaining dynamic square formations and performing precision payload delivery using coordinated control. The system integrates physics-based modeling, formation algorithms, PID-based stabilization, and Lua-Python interfacing in CoppeliaSim to achieve realistic and repeatable outcomes.

Test results validate that the formation control and bomb-dropping logic are reliable under a range of simulated disturbances and failures. Furthermore, this work provides a strong foundation for real-world extensions involving onboard sensing, navigation, and collaborative autonomy.

ACKNOWLEDGMENT

This project was carried out under the mentorship of Dr. Shri Vishal Tripathi, whose technical guidance, critical feedback, and continuous support played a pivotal role in shaping the direction and success of this work. We are grateful for the access to laboratory resources and simulation tools provided by the Department of Electronics and Communication Engineering, IIIT Naya Raipur.

REFERENCES

- [1] Coppelia Robotics, "CoppeliaSim User Manual," 2024. [Online]. Available: <https://www.coppeliarobotics.com>
- [2] J. Shi and Y. Sun, "Multi-agent formation control: A review," *IEEE Transactions on Industrial Electronics*, vol. 69, no. 2, pp. 1103–1115, 2022.
- [3] S. Bouabdallah and R. Siegwart, "Backstepping and Sliding-mode Techniques Applied to an Indoor Micro Quadrotor," in *Proc. IEEE Int. Conf. on Robotics and Automation*, 2005.
- [4] M. Mesbahi and M. Egerstedt, *Graph Theoretic Methods in Multiagent Networks*, Princeton University Press, 2010.
- [5] R. Olfati-Saber, "Flocking for Multi-Agent Dynamic Systems: Algorithms and Theory," *IEEE Transactions on Automatic Control*, vol. 51, no. 3, pp. 401–420, 2006.